

Software Architectures

DE52

Yazan MUALLA

CIAD, Univ. Bourgogne Franche-Comté, UTBM,
France

- o **Introduction**
- o Kruchten's 4 + 1 Model View
- o Architectural Styles
 - 1. Language-based Systems
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems
 - 4. Layered Systems
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems
 - 7. Process Control Systems
- o Service-Oriented Architecture

Architecture Overview and Process

- The underlying design of a software system is called **software architecture**
- It examines the system from a variety of angles at the **highest level**
- It specifies which elements are included in the system, what functions they serve, and how they interact with one another
- In a nutshell, it's the **large picture** or **overall structure** of the entire system, as well as how everything interacts

Architecture Overview and Process

- Architectural Styles and Patterns:
 - Language-based Systems
 - Interpreter-based Systems
 - Repository-based Systems
 - Layered Systems
 - Implicit Invocation Systems
 - Dataflow Systems
 - Process Control Systems

- Quality Attributes (Non-functional requirements):
 - Scalability
 - Performance
 - Security
 - Maintainability
 - Availability
 - Modifiability

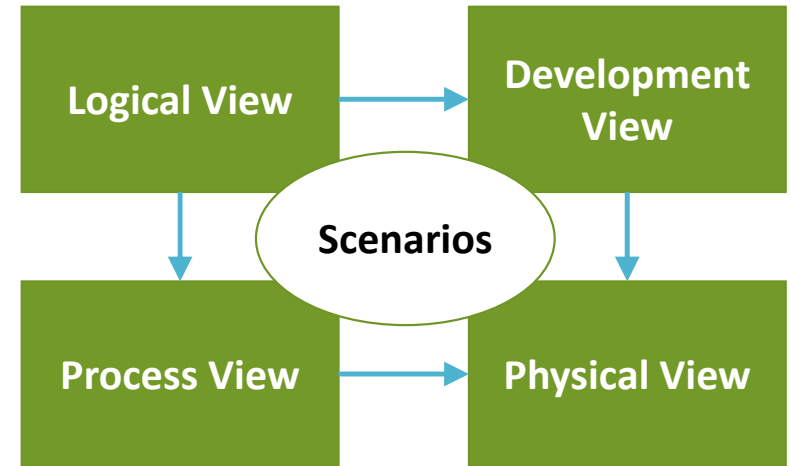
Architecture Overview and Process

- Architectural Decisions:
 - Documented choices on tools, frameworks, protocols, etc., based on trade-offs
- Architecture Description Languages (ADLs):
 - Formal languages like Acme, AADL, or UML for specifying architectures
- Tools and Frameworks
 - **Modeling:** UML, ArchiMate
 - **Documentation:** C4 Model, Kruchten's 4+1 View Model

- o Introduction
- o **Kruchten's 4 + 1 View Model**
- o Architectural Styles
 - 1. Language-based Systems
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems
 - 4. Layered Systems
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems
 - 7. Process Control Systems
- o Service-Oriented Architecture

Kruchten's 4 + 1 View Model

- Philippe Kruchten presented a method for deciphering a system's design
- The four views (logical, process, development, and physical) plus one (Scenarios)
- The four views correspond to the important **viewpoints** about designing a software solution.



Logical view

- The logical view is primarily concerned with meeting a **system's functional requirements**
- In practice, the logical view usually focuses on the system's objects. You can make a **UML class diagram** out of these objects to show the logical view
- When establishing database designs, class diagrams are also useful for defining the logical view

Process view

- The process view focuses on achieving **nonfunctional requirements** that define the system's desirable qualities
- These include quality attributes such as performance and availability
- The process view, on top of these nonfunctional needs, displays processes that relate to the objects in a logical view. The process view, for example, would display the order in which your various items were executed (UML sequence diagram)
- A UML sequence diagram would be helpful for illustrating the methods, how they are executed and in what order.

Development view

- The development view describes the **hierarchical software structure**
- Programming languages, libraries, and tool sets are all considered in the development view
- This includes, in addition to code, management elements such as **scheduling, budgets, and work assignments**
- In terms of development, project management is critical

Physical view

- The physical view controls how items in the logical process are translated to different nodes or **hardware** for the system to execute
- A UML deployment diagram can show how components of a system are distributed across hardware and execution environments

Scenarios

- Scenarios correspond to **system's use cases** or **user tasks** and demonstrate how the four other views interact
- A script outlines the sequence of interactions between objects and processes for each scenario

Note: The 4 + 1 view model is not required for all software architectures, and If any of the views is thought to be useless they can be omitted.

- o Introduction
- o Kruchten's 4 + 1 Model View
- o **Architectural Styles**
 - 1. **Language-based Systems**
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems
 - 4. Layered Systems
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems
 - 7. Process Control Systems
- o Service-Oriented Architecture

1.1. Object-Oriented

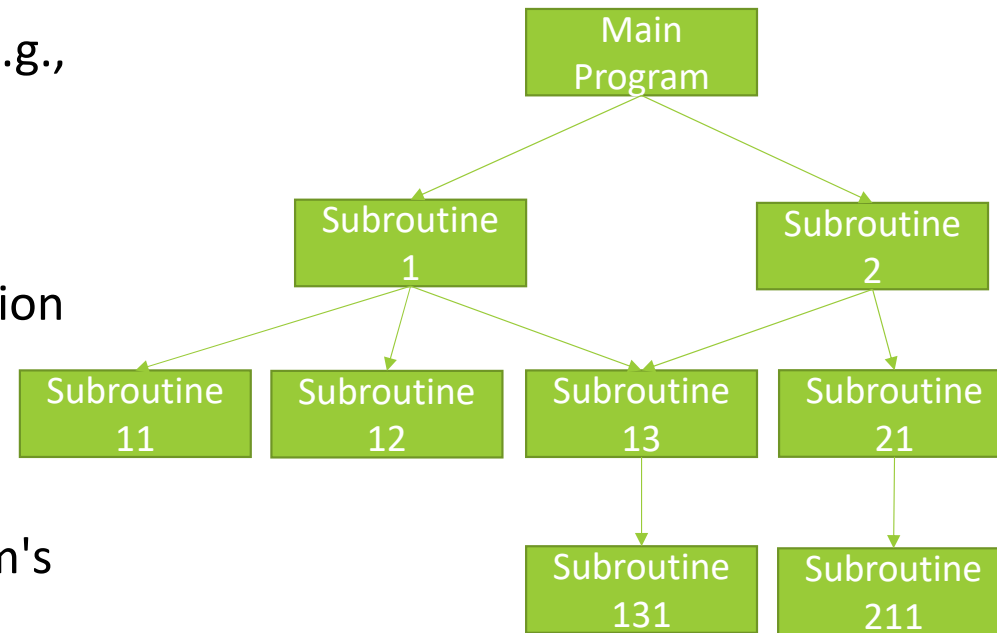
- Object-oriented architectural style results from the object-oriented programming paradigm
- The term "object-oriented" refers to a system made up of objects, each of which is an instance of a class
- Objects can communicate with one another by using their methods
- Inheritance among abstract data types is possible using the object-oriented paradigm

1.1. Object-Oriented

- Object-oriented principles :
 - Abstraction
 - Encapsulation
 - Decomposition
 - Generalization
- Object-oriented design pattern categories :
 - Creational
 - Structural
 - Behavioral
- The object-oriented programming paradigm, despite its widespread use, will not always be the best answer to a problem

1.2. Main Program and Subroutine

- The **procedural programming paradigm**, e.g., C, has a main program and subroutine architectural style
- The phrases routine, procedure, and function are interchangeable when referring to a subroutine
- This style **focuses on functions**: The system's overall operation is divided into a main program and subroutines
- The resulting code has a **hierarchical** structure rather than being flat. Represented as a directed graph. Arrows are subroutine calls



1.2. Main Program and Subroutine

- **Variables** are used to store data in this paradigm
- **Abstract data types** are supported by procedural programming
- **Inheritance is not expressly supported**
- The behavior of functions and how data passes through them are the most important considerations. As a result, it is appropriate for systems that focus on **computing**

1.2. Main Program and Subroutine

- **Local variables** can be defined for each procedure. All data inside the scope of a function is accessible
- Data can be passed as parameters, **by value**, or **by reference** to the procedure. Data can also be passed as a return value from a function
- **Modularity** and **function reuse** are encouraged in this architectural design
- **Drawback:** During execution, data changes made by another subroutine may influence a subroutine. For example, global data that is shared among subroutines

- o Introduction
- o Kruchten's 4 + 1 Model View
- o **Architectural Styles**
 - 1. Language-based Systems
 - 2. Interpreter-based Systems**
 - 3. Repository-based Systems
 - 4. Layered Systems
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems
 - 7. Process Control Systems
- o Service-Oriented Architecture

2.1. Interpreters

- End users can **write scripts, macros, or rules** that access, compose, and operate the basic functionality of those systems in novel and dynamic ways using interpreter-based systems. For example, a formula in Microsoft Excel
- Because it can interpret the formula into an **intermediate representation** and subsequently execute that representation, the application can do the intended calculation
- Because it provides users with **flexibility** and **portability**, the interpreter-based architecture is employed in several commercial systems

2.1. Interpreters

- **Macros** are a type of script that gained popularity after the development of graphical user interfaces. A macro is a program that records keyboard and mouse input and then executes it later
- A **web browser extension**, for example, is a component that enhances the browser's functionality. A component like this is often written in a language like Javascript and interpreted by a browser interpreter
- With the rise of virtual computers and virtual environments, portability coming from Interpreters is becoming increasingly crucial
- **Drawback:** interpreters might be slow. This is a basic trade-off: an interpreted language may be faster and more flexible for developers, but it is slower for the computer to execute

Key Differences Between Compiler and Interpreter

Feature	Compiler	Interpreter
Translation Method	Translates the entire program at once into machine code	Translates and executes line by line or instruction by instruction
Output	Produces an executable file (e.g., .exe)	No separate output ; executes the code directly
Speed	Faster execution after compilation	Slower execution , especially for large programs
Error Detection	Shows all errors after compiling the full code	Shows errors one at a time , during execution
Examples	C, C++, Go, Rust	Python, Ruby, JavaScript, MATLAB
Memory Usage	Generally more efficient	May use more memory during execution
Reusability	Compiled code can be run multiple times without recompilation	Interpreted code must be re-interpreted each time

Java uses both a compiler and an interpreter

- Java Compiler (javac)
 - Translates the Java source code (.java files) into bytecode (.class files)
 - Bytecode is platform-independent, meaning it can run on any system with a Java Virtual Machine (JVM)
 - This step is compilation
- Java Virtual Machine (JVM)
 - The JVM interprets the bytecode at runtime
 - Initially, JVM used a pure interpreter, but now it uses Just-In-Time (JIT) Compiler: Converts bytecode to native machine code during execution for better performance
 - This step is interpretation and runtime compilation

2.2. State Transition Systems

- A state transition system is a term used to describe **how a system behaves**
- Terminology
 - **State:** defined by the information it remembers
 - **Transition:** used to describe the change of a system from one state to another. Multiple transitions can exist inside a same system state. This makes the behaviors non-deterministic
- **Labeled** or **unlabeled** state transition systems exist. For systems where the transition between all states is the same, the states are unlabeled
- **Directed graphs** can be used to describe state transition systems

2.2. State Transition Systems

- **Uses:**

- Consider an operating system and how it must manage the distribution of resources for a large number of processes. A state transition system can assist us in determining **when crucial system events occur**, such as the use of resources, the processes that will consume resources, and the length of time they will be used
- State transition systems can assist us understand how **parallel processing**, **multithreading**, and **distributed computing** affect our system's overall state
- We can use state transition systems to identify **deadlocks**. When one process is waiting forever for another to release a shared resource or finish its job, it is said to be in a deadlock

- o Introduction
- o Kruchten's 4 + 1 Model View
- o **Architectural Styles**
 - 1. Language-based Systems
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems**
 - 4. Layered Systems
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems
 - 7. Process Control Systems
- o Service-Oriented Architecture

3.1. Databases

- Information must be shared across different components in modern software systems
- **The necessity of storing information:** a component's state should not be only temporary while it is running
- **Files are not a good option:** file format and distributing the file to all the components. Furthermore, components may not need all the data that is stored in the files
- A **data-centric software** will not only allow you to store and transfer data or part of it across many components, but it will also improve the system's maintainability, reusability, and scalability

3.1. Databases

- There are two sorts of components at the heart of a data-centric architecture:
 - **Central data:** stores and serves data to all other components connected to it,
 - **Data accessories:** connect to the central data component
- The central data is stored in a database. Two key qualities are considered:
 - **Data integrity:** ensure the data is accurate and consistent over its lifespan
 - **Data persistence:** ensure that data will continue to live on after a process has been terminated
- **Relational databases** that use tables are one type of database that can store data. To query or ask the database for information, relational databases employ Structured Query Language (SQL)
- A Database Management System (DBMS) can automate the management and optimization of queries and transactions

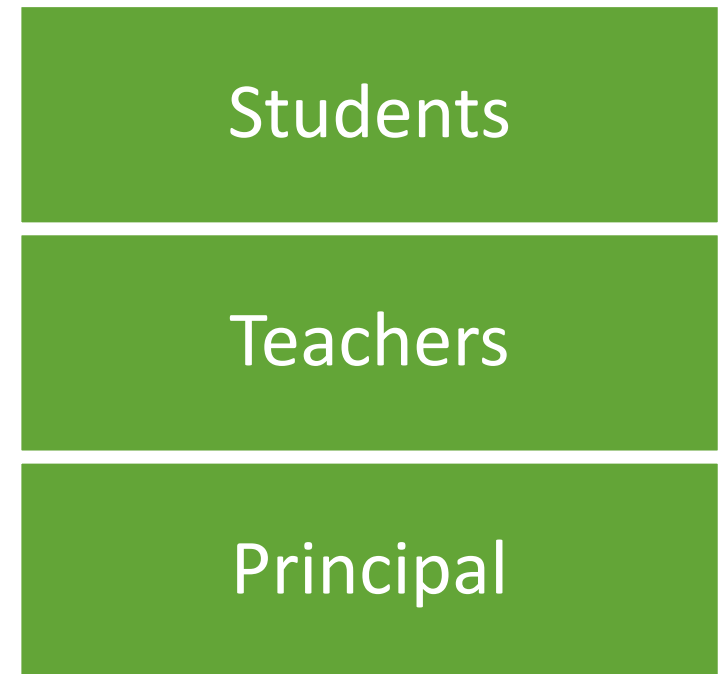
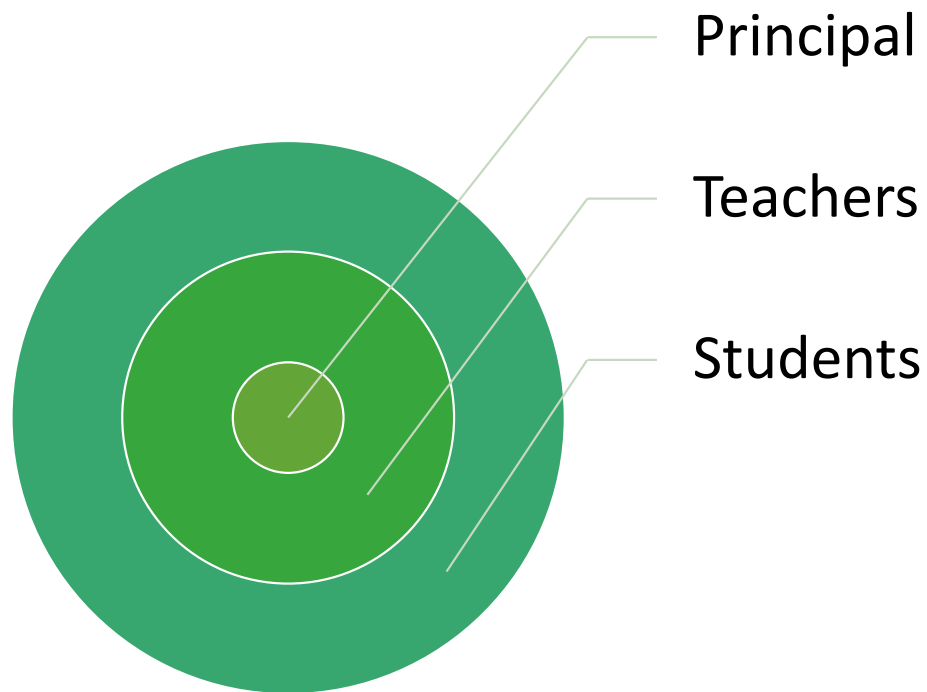
3.1. Databases

- **Drawbacks:**

- **The system becomes overly reliant on the central data component.** Your entire system will be affected if the data server goes down, becomes unusable, or includes corrupted data. Data redundancies, which repeat your data onto separate hard disks, are among the precautions in place
- All data accessories are dependent on data stored in the database. If you're adding data accessories to use a preexisting database, then you will need to build around the existing data schema
- Changing the actual data schema is difficult and will have an impact on your data accessories

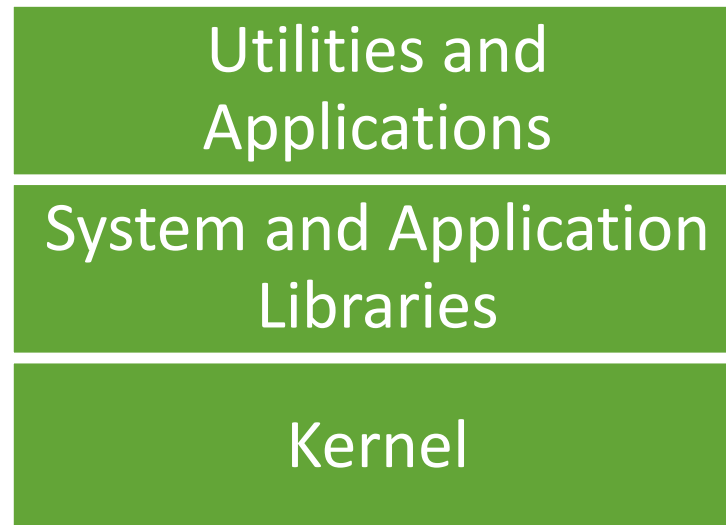
- o Introduction
- o Kruchten's 4 + 1 Model View
- o **Architectural Styles**
 - 1. Language-based Systems
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems
 - 4. Layered Systems**
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems
 - 7. Process Control Systems
- o Service-Oriented Architecture

4.1. General Layered Systems



4.1. General Layered Systems

- A layer's components only interact with those in their own layer or neighbouring layers
- Layering allows the **separation of concerns** into several layers. Many layered systems are divided into three layers: **presentation, logic, and data**

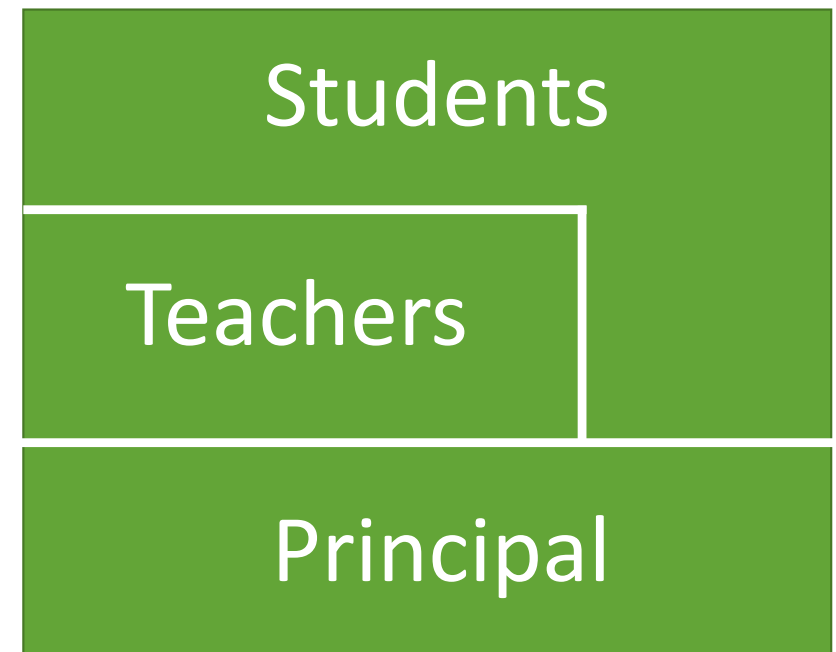


4.1. General Layered Systems

- **Benefits:**
 - Users may perform complex tasks without understanding the layers below
 - The multiple layers can be performed at different degrees of **authorization or privilege**
 - Design will be more **loosely coupled**
- Not every software issue can be transferred to a layered architecture with ease
- **Efficiency tradeoff:** Any real system will have some interactions that are simply transmitted from one layer to the next. This additional communication adds complexity to the software and consumes processing resources

4.1. General Layered Systems

- Allowing for **pass-through** can relax the layered architecture. However, your program will be more **tightly coupled** as a result of this
- In some layered systems, a layer may contain more than one grouping

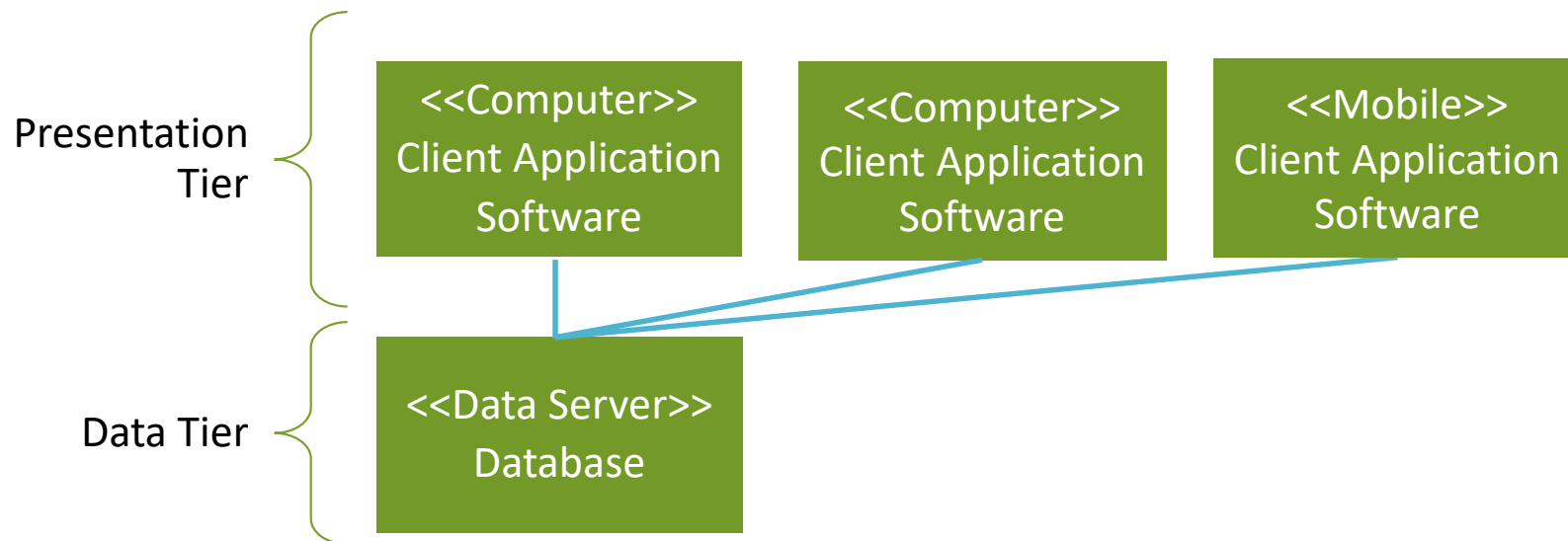


4.2. Client Server n-Tier

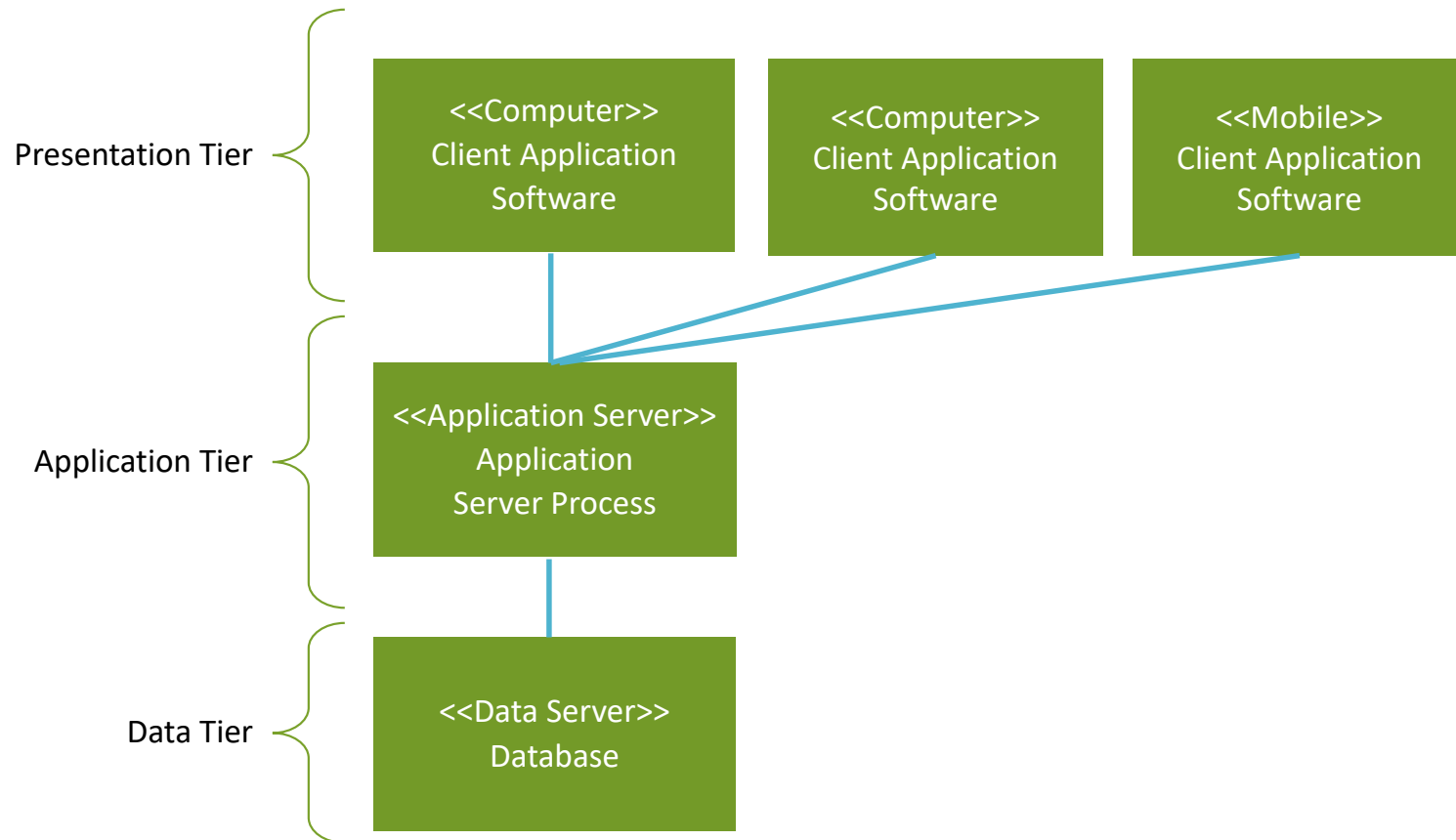
- Tiers are comparable to layers, although they typically relate to components on different physical machines
- The number of tiers can vary greatly; 3- and 4-tier systems are typical, but any number can be used
- In 2-tier design, the relationship between the two tiers is frequently a client/server relationship:
 - Services are provided by the server side. This could be anything from storing data in a database to doing calculation activities
 - These services are requested by the client via messages. **Request-Response** is the name of the communication between the two parties
 - A tier can serve as both a client and a server

4.2. Client Server n-Tier

- Client-Host (Presentation Tier) and Server-Host (Data Tier) are terms used to describe machines that host client and server software, respectively



4.2. Client Server n-Tier



4.2. Client Server n-Tier

■ Advantages

- **Scalability:** As long as the server can handle all the requests in a reasonable amount of time, new clients can be added, and requests can be made
- The advantages of centralizing functionality on a server are evident

■ Disadvantages

- It needs additional resources to manage client-server connections: There are more machines or processes to manage when you add more layers. The added variety and complexity of numerous communication protocols across machines makes the system more difficult to update or maintain
- A server serves as a single point of failure in a network. It is possible to have redundant servers, however this adds complexity

- o Introduction
- o Kruchten's 4 + 1 Model View
- o **Architectural Styles**
 - 1. Language-based Systems
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems
 - 4. Layered Systems
 - 5. Implicit Invocation Systems**
 - 6. Dataflow Systems
 - 7. Process Control Systems
- o Service-Oriented Architecture

5.1. Event Based

- The **event-driven programming paradigm** inspired the event-based architectural style
- The system's core elements are **events**. **Signals, user inputs, messages, or data** from other processes or **programs** are all examples of events
- Events serve as both **indicators of system change** and **function triggers**. Functions in this paradigm are represented by **event generators** and **event consumers**
- Event generators send events, which are received and processed by event consumers. A function can be both a generator and a consumer of events

5.1. Event Based

- An **event bus** connects all event generators and consumers in the system, allowing them to communicate with one other
- Bind an event and an event consumer using the event bus. To be notified about specific occurrences, each event consumer registers with the event bus. When an event is detected by the event bus, it is distributed to all eligible event consumers
- The event-based architectural style is reflected through the **observer design pattern**

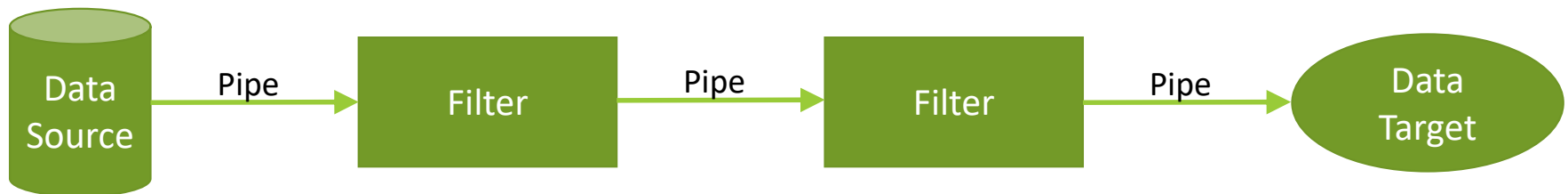
5.2. Publish-Subscribe

- The publish-subscribe architectural style, like the event-based architectural style, relies on implicit invocation. **Publishers** and **subscribers** are not explicitly linked, so they have a loose coupling. Publishers send messages, while subscribers can subscribe or unsubscribe to receive these messages.
- The loose relationship between publishers and subscribers allows for scalability and flexibility in publish-subscribe systems
- It is very difficult to change a message that has already been sent. A correction message is the sole means to communicate a change
- RSS feeds and mobile messaging subscriptions are two examples

- o Introduction
- o Kruchten's 4 + 1 Model View
- o **Architectural Styles**
 - 1. Language-based Systems
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems
 - 4. Layered Systems
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems**
 - 7. Process Control Systems
- o Service-Oriented Architecture

6.1. Pipes and Filters

- Pipe and filter architecture is a type of **data flow architecture**
- A data flow architecture views a system to be nothing more than a **sequence of data transformations**. Different sorts of operations are used to change data from one form to another
- A pipe and filter architecture consists of independent entities known as **filters that execute transformations on data input** and **pipes that serve as connections for the stream of data** being modified



6.1. Pipes and Filters

■ Advantages

- Ensures loose and flexible coupling of components (the filters)
- Filters can be referred to as “black boxes”
- Reusability

■ Disadvantages

- **Filter overheads reduce performance.** Each filter in this design will receive data, parse it into a data structure, do transformations, and finally send the data out. That's a lot of overhead if each filter has to go through this process with comparable processing at the output stages
- Filters become **overloaded** as a result of the large amounts of data they must handle. While one filter is working hard to process the large amount of data, other filters may be starved waiting for their inputs.
- It is **unsuitable for interactive applications**

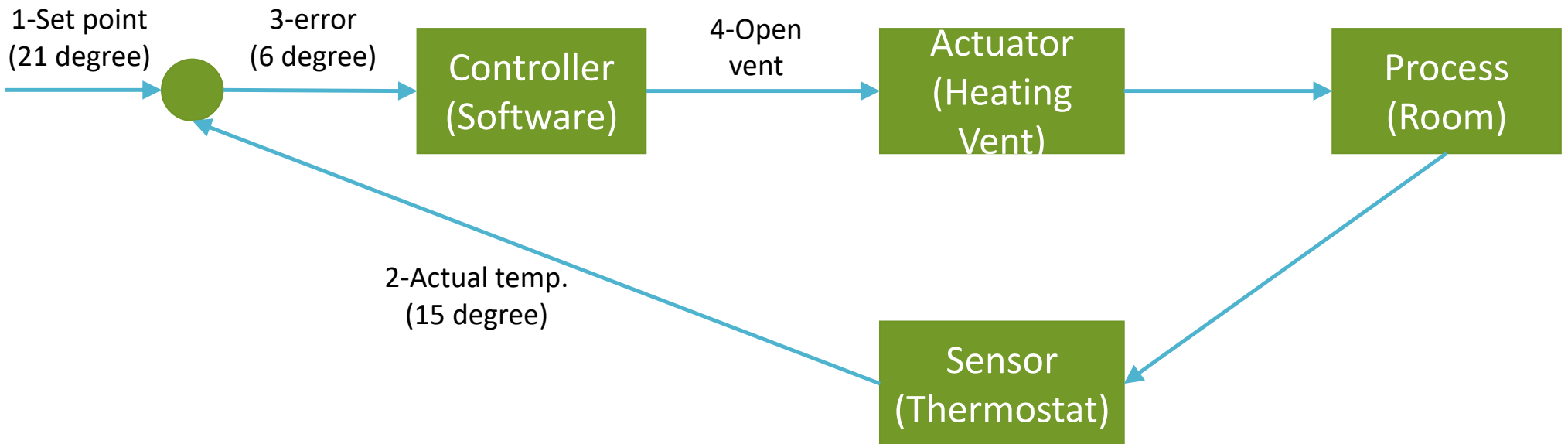
- o Introduction
- o Kruchten's 4 + 1 Model View
- o **Architectural Styles**
 - 1. Language-based Systems
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems
 - 4. Layered Systems
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems
 - 7. Process Control Systems**
- o Service-Oriented Architecture

7.1. Process Control

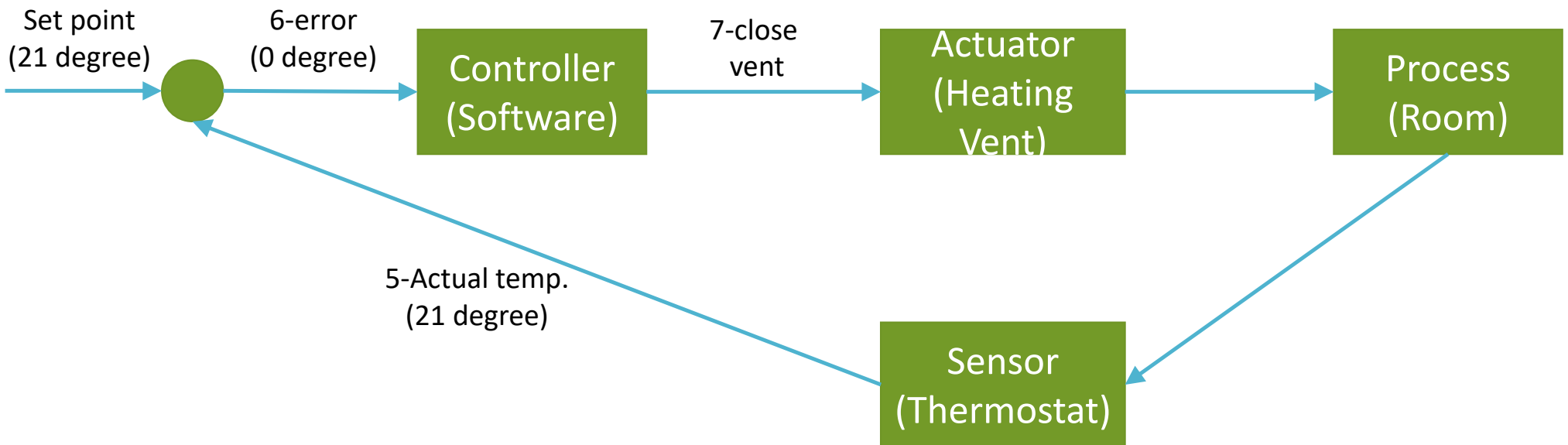
- All kinds of industrial processes are controlled by software. Many operations require process control to run efficiently and safely. A **feedback loop** is the most fundamental form of process control
- A straightforward method for controlling a room in the real world. There is a heating vent and a thermostat in this room. The goal is to keep the room at a comfortable temperature
- There are four main components to a feedback loop:
 - **Sensor**: monitors some crucial data (e.g., a thermostat);
 - **Controller**: logic (e.g., software);
 - **Actuator**: the physical technique of controlling the process (e.g., a heating vent);
 - **Process**: what you're trying to regulate (e.g., room)

7. Process Control Systems

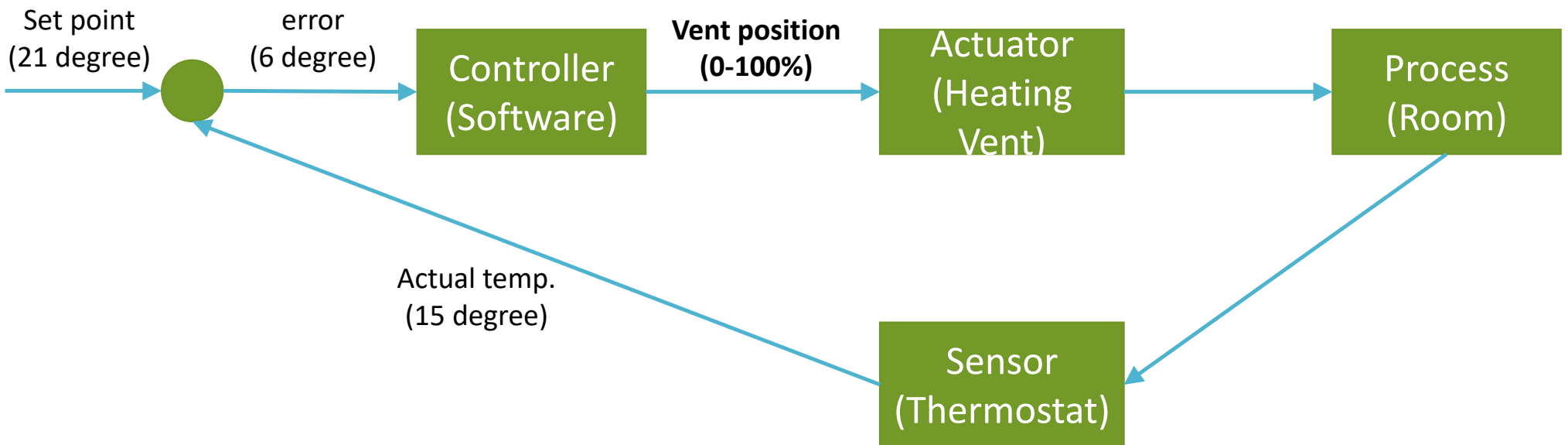
7.1. Process Control (Part 1)



7.1. Process Control (Part 2)



7.1. Process Control (Improved)



- o Introduction
- o Kruchten's 4 + 1 Model View
- o Architectural Styles
 - 1. Language-based Systems
 - 2. Interpreter-based Systems
 - 3. Repository-based Systems
 - 4. Layered Systems
 - 5. Implicit Invocation Systems
 - 6. Dataflow Systems
 - 7. Process Control Systems
- o **Service-Oriented Architecture**

What Is Service-Oriented Architecture?

- Service-Oriented Architecture (SOA) is a software design paradigm in which application components deliver services to other components over a network through a communication protocol
- Several services connect with each other in a service-oriented architecture in one of two ways:
 - Passing data
 - Two or more services coordinating an activity
- **Web services** are commonly used to implement SOA, making the functional building blocks accessible over standard internet protocols

Service-Oriented Architecture Patterns

- Each of the SOA building pieces has three roles:
 - Service provider
 - Service broker, service registry, service repository
 - Service requester/consumer
- The service provider collaborates with the service registry to establish **why** and **how** certain services, such as security and availability, are provided
- The service requester **locates entries** in the broker registry and then **binds** them to the service provider. They could (or not) access multiple services; that depends on the capability of the service requester

Implementation: Simple Object Access Protocol (SOAP)

- Simple Object Access Protocol (SOAP) is a messaging protocol specification for exchanging **structured** information in the implementation of web services in computer networks
- SOAP is a **standard protocol** that was created to allow applications written in various languages and on various platforms to communicate with one another
- Because it's a protocol, it has **built-in restrictions** that add to its complexity and overhead, **resulting in longer page load times**. These standards, on the other hand, include **built-in compliances** that may make them more suitable for **enterprise applications**
- **Security**, in addition to Atomicity, Consistency, Isolation, and Durability (**ACID**), which is a set of attributes for assuring safe database transactions, are built-in compliance standards

Implementation: Representational State Transfer (REST)

- REST is a **set of architectural principles** tailored to the requirements of **lightweight** web services and mobile apps
- It's a set of guidelines, so it's up to developers to put the guidelines into action
- A request for data is often sent to a REST API via the hypertext transfer protocol (HTTP). APIs created for REST (also known as RESTful APIs or RESTful web services) can return messages in HTML, XML, plain text, and JSON after receiving a request
- JSON (JavaScript object notation) is a popular messaging format because:
 - can be read by any programming language,
 - human-readable and machine-readable,
 - Lightweight

Advantages

- **Simple to Maintain:** Any service developed using the SOA architecture is simple to edit and update. You won't have to upgrade your system because the service is maintained by a third party
- **Improved Quality of Code:** Because services run freelance of the system, they have their own code
- **Platform Independence:** Services connect with other apps using a common language, implying that they are independent of the platform on which they are executing
- **Scalable:** When a service gets a lot of users, it's usually only a matter of adding more servers
- **Independent of Other Services:** Services created using SOA principles are independent of each other

Disadvantages

- **High Bandwidth Server:** Because some web services send and receive messages and information regularly, a million requests per day is not uncommon. To run a web service, you'll need a high-speed server with plenty of data capacity
- **Extra Overload:** Requests are validated prior to being transmitted to the service. If you use many services, your system will get overloaded with additional computing
- **Somehow High Cost:** In terms of human resources, development, and technology, it is costly

References and Acknowledgments

- UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design
- Design Patterns: Elements of Reusable Object-Oriented Software
- Timeless Way of Building, Alexander, ISBN 0-19-502402-8
- Software Design and Architecture by Kenny Wong, University of Alberta Computing Science, Faculty of Science, Coursera
- A Pattern Language, Alexander, 0-19-501-919-9
- Design Patterns, Gamma, et al., 0-201-63361-2 CD version 0-201-63498-8
- Pattern-Oriented Software Architecture, Vol. 5, Buschmann, et al., 0-471-48648-5
- Concurrent Programming in Java, 2nd ed., Lea, 0-201-31009-0
- Applying UML & Patterns, 2nd ed., Larman, 0-13-092569-1
- "Object-Oriented Patterns," P. Coad; Comm. of the ACM, 9/92
- "Documenting Frameworks using Patterns," R. Johnson; OOPSLA '92
- <https://medium.com/@SoftwareDevelopmentCommunity/what-is-service-oriented-architecture-fa894d11a7ec>
- <https://www.educba.com/what-is-soa/>
- [https://www.redhat.com/en/topics/integration/whats-the-difference-between-soap-rest#:~:text=Representational%20state%20transfer%20\(REST\)%20is,protocol%20while%20REST%20is%20not.](https://www.redhat.com/en/topics/integration/whats-the-difference-between-soap-rest#:~:text=Representational%20state%20transfer%20(REST)%20is,protocol%20while%20REST%20is%20not.)
- <https://www.itrelease.com/2018/10/advantages-and-disadvantages-of-service-oriented-architecture-soa/>
- Aranda, Jorge, Ramzan Khuwaja, and Steve Easterbrook. "Discovering the shared understanding dynamics of large software teams." Proceedings of the 2007 conference of the center for advanced studies on Collaborative research. 2007.

End of Lecture